

AADK 7: Task 1 — State Concepts Deep Dive & Visual Mapping

Student Name: Rajat Prakash Dhal

Course/Unit: Android Development with Kotlin – Session 7

USN: 1hk22cs115

email: 1hk22cs115@hkbk.edu.in

1. Key Terms (Definitions with Examples)

State

State refers to any data that can change over time and affects what is displayed on the UI. When the state changes, Jetpack Compose automatically updates the UI to reflect the new value.

Example (Pseudo Code):

```
var count by remember { mutableStateOf(0) }
```

```
Button(onClick = { count++ }) {  
    Text("Count: $count")  
}
```

When the button is clicked, the state variable count changes and the UI updates automatically.

remember()

remember() is used to store a value in memory during recompositions. It helps preserve the value as long as the composable remains in the composition.

Example:

```
val name = remember { mutableStateOf("") }
```

Here the value of name is remembered even when the UI recomposes.

mutableStateOf()

mutableStateOf() creates a mutable observable state. When the value changes, Compose triggers recomposition to update the UI.

Example:

```
var isChecked by remember { mutableStateOf(false) }
```

When isChecked changes, UI elements that depend on it update automatically.

State Hoisting

State hoisting is the practice of moving state to a higher-level composable so that multiple child composables can share and control it.

Example:

```
@Composable
```

```
fun Parent() {  
    var value by remember { mutableStateOf("") }  
    Child(value, { value = it })  
}
```

```
@Composable
```

```
fun Child(text: String, onTextChange: (String) -> Unit) {  
    TextField(value = text, onValueChange = onTextChange)  
}
```

Here the state is stored in the parent and passed to the child.

Recomposition

Recomposition is the process where Compose redraws parts of the UI when the state changes.

Example:

```
var counter by remember { mutableStateOf(0) }
```

```
Text("Counter: $counter")  
  
Button(onClick = { counter++ }) {  
    Text("Increase")  
}
```

When counter changes, Compose recomposes and updates the displayed value.

2. Interaction Scenario

Scenario: Tip Calculator Interaction

User enters a bill amount → tip is calculated → user can toggle switch to round up tip → final amount displayed.

State Variables

- billAmount = ""
- tipAmount = 0.0
- roundUp = false

Flow Diagram (Text Representation)

User enters bill amount



State Variable Updated

billAmount = "100"



Tip Calculation

tipAmount = billAmount * 0.15



User toggles round-up switch

roundUp = true



State Change Applied

tipAmount rounded to nearest value



UI Recomposition

Updated tip displayed on screen

State Transitions Example

Action	State Change
User types bill	billAmount changes
Tip calculated	tipAmount updated
Toggle switch ON	roundUp: false → true
UI updates	recomposition triggered

3. State Handling Strategy Table

UI Component	State Variable	Stateful or Stateless?	Where is State Defined?	Comments
TextField (Bill Amount)	billAmount	Stateful	Parent Composable	Stores user input
Switch (Round Up Tip)	roundUp	Stateful	Parent Composable	Determines rounding behavior
Text (Tip Result)	tipAmount	Stateless	Calculated from state	Displays computed result

4. Reflection

State hoisting is important because it allows better control and sharing of state between composables. When state is lifted to a parent composable, multiple UI components can access and update it consistently. If state is deeply nested inside child composables, it becomes harder to manage and reuse components. This can also cause unexpected UI behavior when different parts of the UI depend on the same data.

A real-world bug from mismanaged state could occur if the toggle switch updates the rounding state locally but the tip calculation uses another state variable. In this case, the UI might not update correctly when the switch is toggled, causing the wrong tip value to be displayed.